

系统开发工具实验报告 3

姓名：李博文 学号：25020007057

第九题

```
#1. 创建目录结构
mkdir -p q09/src/greetlab
cd q09

#2 新建 __init__.py
touch src/greetlab/__init__.py

#3 写 cli.py
cat > src/greetlab/cli.py <<'EOF'
import argparse

def main():
    p = argparse.ArgumentParser()
    p.add_argument("--name", required=True)
    a = p.parse_args()
    print(f"Hello, {a.name}!")
EOF

#4. 写 pyproject.toml
cat > pyproject.toml <<'EOF'
[build-system]
requires = ["setuptools>=68"]
build-backend = "setuptools.build_meta"

[project]
name = "greetlab-25020007057"
version = "0.1.0"

[project.scripts]
sdt-greet = "greetlab.cli:main"
EOF

#5. 创建打包用虚拟环境
python3 -m venv buildenv
```

```

#激活环境
source buildenv/bin/activate

#6 在虚拟环境内安装build
pip install build

#7 执行打包, 生成wheel
python -m build

#8 退出打包环境
deactivate

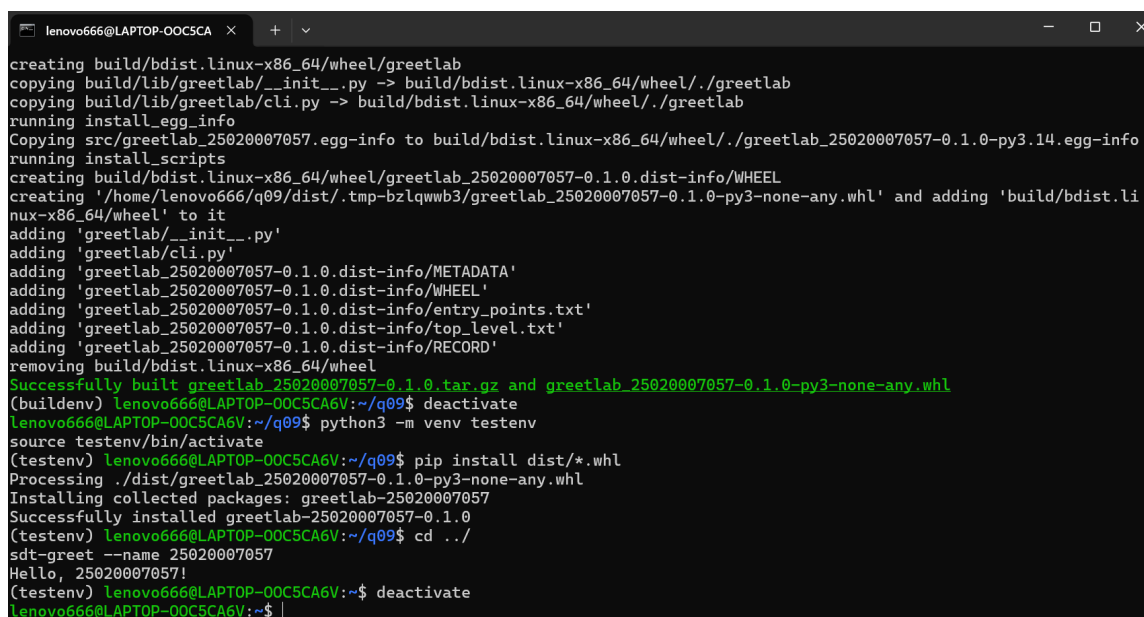
#9 创建干净虚拟环境 testenv
python3 -m venv testenv
source testenv/bin/activate

#10 在 testenv 里, 安装打包好的 wheel
pip install dist/*.whl

#11 切换到 q09 文件夹外面测试
cd ../
#执行命令
sdt-greet --name 25020007057

#12 测试完毕, 退出虚拟环境
deactivate

```



```

lenovo666@LAPTOP-OOC5CA  x + v
creating build/bdist.linux-x86_64/wheel/greetlab
copying build/lib/greetlab/__init__.py -> build/bdist.linux-x86_64/wheel/./greetlab
copying build/lib/greetlab/cli.py -> build/bdist.linux-x86_64/wheel/./greetlab
running install_egg_info
Copying src/greetlab_25020007057.egg-info to build/bdist.linux-x86_64/wheel/./greetlab_25020007057-0.1.0-py3.14.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/greetlab_25020007057-0.1.0.dist-info/WHEEL
creating '/home/lenovo666/q09/dist/.tmp-bz1qwwb3/greetlab_25020007057-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'greetlab/__init__.py'
adding 'greetlab/cli.py'
adding 'greetlab_25020007057-0.1.0.dist-info/METADATA'
adding 'greetlab_25020007057-0.1.0.dist-info/WHEEL'
adding 'greetlab_25020007057-0.1.0.dist-info/entry_points.txt'
adding 'greetlab_25020007057-0.1.0.dist-info/top_level.txt'
adding 'greetlab_25020007057-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built greetlab_25020007057-0.1.0.tar.gz and greetlab_25020007057-0.1.0-py3-none-any.whl
(lenv) lenovo666@LAPTOP-OOC5CA6V:~/q09$ deactivate
lenovo666@LAPTOP-OOC5CA6V:~/q09$ python3 -m venv testenv
source testenv/bin/activate
(testenv) lenovo666@LAPTOP-OOC5CA6V:~/q09$ pip install dist/*.whl
Processing ./dist/greetlab_25020007057-0.1.0-py3-none-any.whl
Installing collected packages: greetlab-25020007057
Successfully installed greetlab-25020007057-0.1.0
(testenv) lenovo666@LAPTOP-OOC5CA6V:~/q09$ cd ../
sdt-greet --name 25020007057
Hello, 25020007057!
(testenv) lenovo666@LAPTOP-OOC5CA6V:~$ deactivate
lenovo666@LAPTOP-OOC5CA6V:~$

```

图 1: 第九题运行截图

第十题

#1. 复制整个 *q09* 目录到 *q10*

```
cp -r ~/q09 ~/q10
```

```
cd ~/q10
```

#2. 写入测试文件 *test_greetlab.py*

```
cat > test_greetlab.py <<'EOF'
```

```
import pytest
```

```
from greetlab.cli import main
```

```
import sys
```

```
def test_blank_name():
```

```
    sys.argv = ["cli.py", "--name", " "]
```

```
    with pytest.raises(SystemExit) as excinfo:
```

```
        main()
```

```
    assert excinfo.value.code == 2
```

```
EOF
```

#3 运行测试，确认失败

#加 *PYTHONPATH=src* 解决找不到模块的问题

```
PYTHONPATH=src pytest test_greetlab.py -v
```

#4 替换 *cli.py*

```
cat > src/greetlab/cli.py <<'EOF'
```

```
import argparse
```

```
def main():
```

```
    p = argparse.ArgumentParser()
```

```
    p.add_argument("--name", required=True)
```

```
    a = p.parse_args()
```

```
    if len(a.name.strip()) == 0:
```

```
        raise SystemExit(2)
```

```
    print(f"Hello, {a.name}!")
```

```
EOF
```

#5 人工查看 *diff*

```
git diff src/greetlab/cli.py
```

#6 再次跑测试

```
PYTHONPATH=src pytest test_greetlab.py -v
```

#7 创建 *ai_log.md* (最多 5 行)

```
cat > ai_log.md <<'EOF'
```

1.提示: *name*仅空白时`main`抛出`SystemExit(2)`, 仅修改`cli.py`, 测试命令`PYTHONPATH=src pytest`

- 2.AI改动：增加strip()校验，空白则raise SystemExit(2)
 - 3.人工diff检查：仅新增判断，无多余修改
 - 4.重跑pytest，测试用例通过
- EOF

```
lenovo666@LAPTOP-OOC5CA  ×  +  v
--skip-to <path>      skip the output to the specified path
--find-object <object-id>
                        look for differences that change the number of occurrences of the specified object
--diff-filter [(A|C|D|M|R|T|U|X|B)...[*]]
                        select files by diff type
--max-depth <depth>    maximum tree depth to recurse
--output <file>        output to a specific file

lenovo666@LAPTOP-OOC5CA6V:~/q10$ diff src/greetlab/cli.py.bak src/greetlab/cli.py
diff: src/greetlab/cli.py.bak: No such file or directory
lenovo666@LAPTOP-OOC5CA6V:~/q10$ cp src/greetlab/cli.py src/greetlab/cli.py.bak
lenovo666@LAPTOP-OOC5CA6V:~/q10$ diff src/greetlab/cli.py.bak src/greetlab/cli.py
lenovo666@LAPTOP-OOC5CA6V:~/q10$ PYTHONPATH=src pytest test_greetlab.py -v
===== test session starts =====
platform linux -- Python 3.14.4, pytest-9.1.1, pluggy-1.6.0 -- /home/lenovo666/.local/share/pipx/venvs/pytest/bin/python
cachedir: .pytest_cache
rootdir: /home/lenovo666/q10
configfile: pyproject.toml
collected 1 item

test_greetlab.py::test_blank_name PASSED [100%]

===== 1 passed in 0.00s =====
lenovo666@LAPTOP-OOC5CA6V:~/q10$ cat > ai_log.md <<'EOF'
1.提示：name仅空白时main抛出SystemExit(2)，仅修改cli.py，测试命令PYTHONPATH=src pytest
2.AI改动：增加strip()校验，空白则raise SystemExit(2)
3.人工diff检查：仅新增判断，无多余修改
4.重跑pytest，测试用例通过
EOF
```

图 2: 第十题运行截图

第十一题

#1 在 q10 目录新建 communication.md

```
cat > communication.md <<'EOF'
```

Issue

环境：Windows（待确认Python版本）

复现命令：sdt-greet --name " "

实际结果：程序输出 Hello, !, 退出码0

期望结果：检测到仅含空白字符的name，以退出码2终止程序

描述：传入仅空格的name参数时，程序未校验空白输入，仍正常打印问候。

提交信息

fix: 增加name空白输入校验

问题：当--name仅传入空白字符，程序不做校验，正常输出问候并返回0。

方案：对name执行strip(), 去除空白后为空则raise SystemExit(2)。

评审意见

Blocking: 代码缺少空白参数校验。

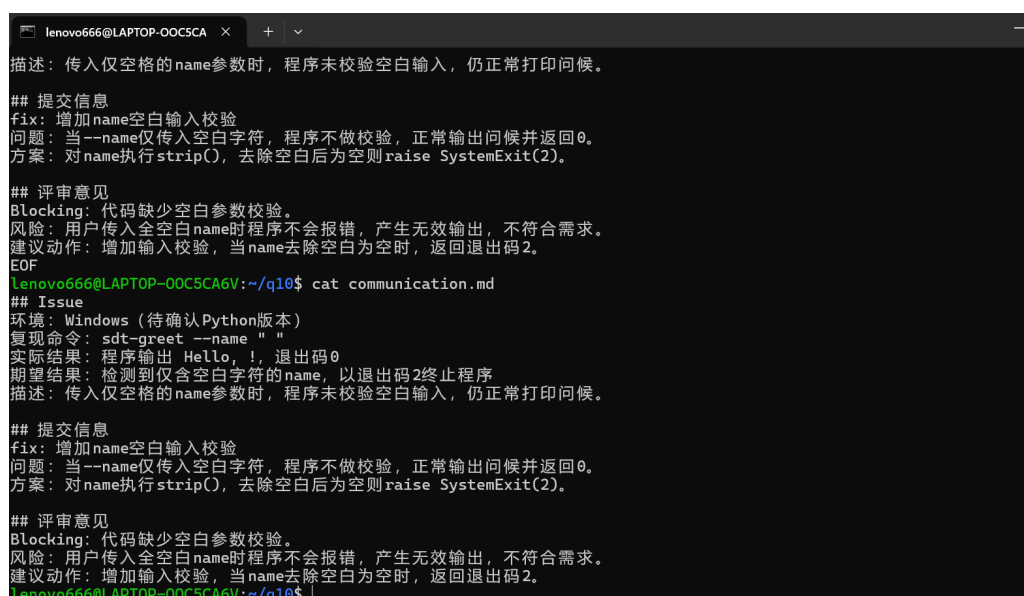
风险：用户传入全空白name时程序不会报错，产生无效输出，不符合需求。

建议动作：增加输入校验，当name去除空白为空时，返回退出码2。

EOF

#2 查看文件确认

cat communication.md



```

lenovo666@LAPTOP-00C5CA  x  +  v
描述：传入仅空格的name参数时，程序未校验空白输入，仍正常打印问候。
## 提交信息
fix: 增加name空白输入校验
问题：当--name仅传入空白字符，程序不做校验，正常输出问候并返回0。
方案：对name执行strip(), 去除空白后为空则raise SystemExit(2)。
## 评审意见
Blocking: 代码缺少空白参数校验。
风险：用户传入全空白name时程序不会报错，产生无效输出，不符合需求。
建议动作：增加输入校验，当name去除空白为空时，返回退出码2。
EOF
lenovo666@LAPTOP-00C5CA6V:~/q10$ cat communication.md
## Issue
环境：Windows (待确认Python版本)
复现命令：sdt-greet --name " "
实际结果：程序输出 Hello, !, 退出码0
期望结果：检测到仅含空白字符的name，以退出码2终止程序
描述：传入仅空格的name参数时，程序未校验空白输入，仍正常打印问候。
## 提交信息
fix: 增加name空白输入校验
问题：当--name仅传入空白字符，程序不做校验，正常输出问候并返回0。
方案：对name执行strip(), 去除空白后为空则raise SystemExit(2)。
## 评审意见
Blocking: 代码缺少空白参数校验。
风险：用户传入全空白name时程序不会报错，产生无效输出，不符合需求。
建议动作：增加输入校验，当name去除空白为空时，返回退出码2。
lenovo666@LAPTOP-00C5CA6V:~/q10$

```

图 3: 第十一题运行截图

第十二题

#1. 创建 q12 目录并进入

```
mkdir ~/q12
```

```
cd ~/q12
```

#2. 写入完整 train.py (补全两处 TODO)

```
cat > train.py <<'EOF'
```

```
import torch
```

```
from torch import nn
```

```
torch.manual_seed(20260907)
```

```
x = torch.linspace(-1, 1, 101).reshape(-1, 1)
```

```

y = 3 * x - 1

model = nn.Linear(1, 1)
loss_fn = nn.MSELoss()
opt = torch.optim.SGD(model.parameters(), lr=0.1)

for _ in range(200):
    pred = model(x)
    loss = loss_fn(pred, y)
    # TODO: 清空梯度、反向传播、更新参数
    opt.zero_grad()
    loss.backward()
    opt.step()

# TODO: 进入评估模式并在no_grad中打印最终损失、weight和bias
model.eval()
with torch.no_grad():
    pred_final = model(x)
    loss_final = loss_fn(pred_final, y)
    w = model.weight.item()
    b = model.bias.item()
    print(f"Final loss: {loss_final:.6f}")
    print(f"Weight: {w:.4f}")
    print(f"Bias: {b:.4f}")
EOF

#3 创建虚拟环境 venv (在 q12 目录)
python3 -m venv venv

#4 激活虚拟环境
source venv/bin/activate

#5 在虚拟环境里安装 CPU 版 PyTorch
pip install torch --index-url https://download.pytorch.org/whl/cpu

#6 运行训练脚本
python train.py

#7 退出虚拟环境
deactivate

```

```

lenovo666@LAPTOP-OOC5CA  x + v
anylinux_2_28_x86_64.whl.metadata (2.7 kB)
Downloading https://download-r2.pytorch.org/whl/cpu/torch-2.14.0%2Bcpu-cp314-cp314-manylinux_2_28_x86_64.whl (196.3 MB)
196.3/196.3 MB 4.7 MB/s eta 0:00:00
Downloading fsspec-2026.7.0-py3-none-any.whl (206 kB)
Downloading networkx-3.6.1-py3-none-any.whl (2.1 MB)
2.1/2.1 MB 12.5 kB/s eta 0:00:00
Downloading https://download.pytorch.org/whl/setuptools-78.1.0-py3-none-any.whl (1.3 MB)
1.3/1.3 MB 2.7 MB/s eta 0:00:00
Downloading sympy-1.14.0-py3-none-any.whl (6.3 MB)
6.3/6.3 MB 717.8 kB/s eta 0:00:00
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
536.2/536.2 kB 2.9 MB/s eta 0:00:00
Downloading typing_extensions-4.16.0-py3-none-any.whl (45 kB)
Downloading filelock-3.32.3-py3-none-any.whl (98 kB)
Downloading https://download.pytorch.org/whl/jinja2-3.1.6-py3-none-any.whl (134 kB)
Downloading https://download.pytorch.org/whl/markupsafe-3.0.3-cp314-cp314-manylinux2014_x86_64.man
ylinux_2_28_x86_64.whl (23 kB)
Installing collected packages: mpmath, typing-extensions, sympy, setuptools, networkx, MarkupSafe, fsspec, filelock, jin
ja2, torch
Successfully installed MarkupSafe-3.0.3 filelock-3.32.3 fsspec-2026.7.0 jinja2-3.1.6 mpmath-1.3.0 networkx-3.6.1 setupto
ols-78.1.0 sympy-1.14.0 torch-2.14.0+cpu typing-extensions-4.16.0
(venv) lenovo666@LAPTOP-OOC5CA6V:~/q12$ python train.py
/home/lenovo666/q12/venv/lib/python3.14/site-packages/torch/_subclasses/functional_tensor.py:368: UserWarning: Failed to
initialize NumPy: No module named 'numpy' (Triggered internally at /__w/pytorch/pytorch/torch/csrc/utils/tensor_numpy.c
pp:84.)
cpu = _conversion_method_template(device=torch.device("cpu"))
Final loss: 0.000000
Weight: 3.0000
Bias: -1.0000
(venv) lenovo666@LAPTOP-OOC5CA6V:~/q12$

```

图 4: 第十二题运行截图

练习

练习 1

将环境保存到一个文件，创建一个 venv，激活它，再激活到另一个文件，然后。环境发生了什么变化？为什么贝壳更喜欢文夫？（提示：观察激活前后。）跑一跑，推理一下停用 bash 功能到底在做什么。 `printenv printenv diff before.txt after.txt $PATH which deactivate`

```

C:\Users\Lenovo>wsl ~
lenovo666@LAPTOP-OOC5CA6V:~$ python3 -m venv venv
lenovo666@LAPTOP-OOC5CA6V:~$ printenv > before.txt
lenovo666@LAPTOP-OOC5CA6V:~$ source venv/bin/activate
(venv) lenovo666@LAPTOP-OOC5CA6V:~$ printenv > after.txt
(venv) lenovo666@LAPTOP-OOC5CA6V:~$ diff before.txt after.txt
10a11
> VIRTUAL_ENV=/home/lenovo666/venv
17a19
> VIRTUAL_ENV_PROMPT=venv
18a21
> PS1=(venv) \[e]0;\u@\h: \w\a\${debian_chroot:+($debian_chroot)}\[\033[01;32m\]\u@\h\[\033[00m\]:\[\033[01;34m\]\w\[\
033[00m\]\$
21c24
< PATH=/home/lenovo666/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:/usr/games:/usr/local/gam
es:/usr/lib/wsl/lib:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/windows/System32/WindowsP
owerShell/v1.0:/mnt/c/windows/System32/OpenSSH:/mnt/d/Windows Kits/10/Windows Performance Toolkit:/mnt/c/Program File
s/NVIDIA Corporation/NVIDIA App/NvDLISR:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Common:/mnt/c/Users/Lenovo/A
ppData/Local/Microsoft/WindowsApps:/mnt/d/vscode/Microsoft VS Code/bin:/snap/bin:/home/lenovo666/.local/bin:/home/lenovo
666/.local/bin
----
> PATH=/home/lenovo666/venv/bin:/home/lenovo666/.local/bin:/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin:
/usr/games:/usr/local/games:/usr/lib/wsl/lib:/mnt/c/windows/system32:/mnt/c/windows:/mnt/c/windows/System32/Wbem:/mnt/c/
windows/System32/WindowsPowerShell/v1.0:/mnt/c/windows/System32/OpenSSH:/mnt/d/Windows Kits/10/Windows Performance Too
lkit:/mnt/c/Program Files/NVIDIA Corporation/NVIDIA App/NvDLISR:/mnt/c/Program Files (x86)/NVIDIA Corporation/PhysX/Com
mon:/mnt/c/Users/Lenovo/AppData/Local/Microsoft/WindowsApps:/mnt/d/vscode/Microsoft VS Code/bin:/snap/bin:/home/lenovo66
6/.local/bin:/home/lenovo666/.local/bin
(venv) lenovo666@LAPTOP-OOC5CA6V:~$ which deactivate
(venv) lenovo666@LAPTOP-OOC5CA6V:~$

```

图 5: 练习 1 截图

1. 修改 PATH: 把 venv/bin 放到 PATH 最前面。之后执行 python、pip, 优先调用 venv 里

的版本，而不是系统 python。

2. 新增环境变量 `VIRTUAL_ENV`，值就是虚拟环境文件夹路径。
3. 定义 shell 函数 `deactivate`（加载进当前 bash shell 内存）。
4. 修改命令提示符 `PS1`，前面加上 `(venv)` 标记，提醒你当前在虚拟环境。

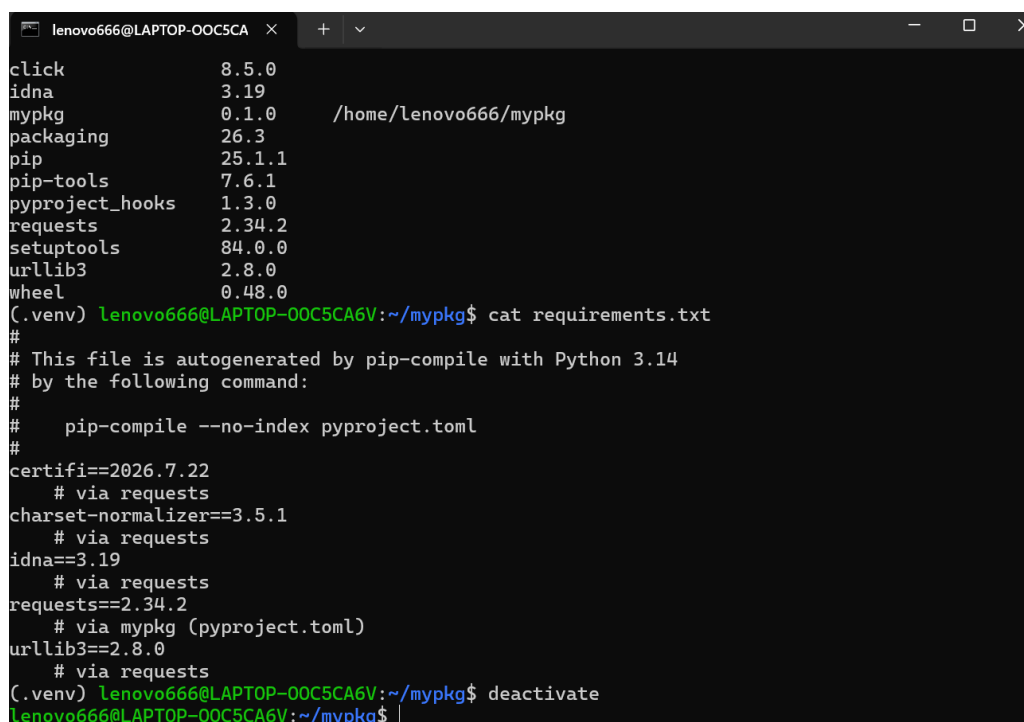
子 shell 里面修改 `PATH`、定义函数，只影响子 shell；子 shell 退出后，所有修改全部丢弃，父 shell 完全不受影响，环境不会激活。

`source activate`（等价. `activate`）：在当前 shell 进程内直接执行脚本，直接修改当前 shell 内存里的变量、注册 `deactivate` 函数，环境生效。

```
printenv > before.txt
python3 -m venv venv
source venv/bin/activate
printenv > after.txt
diff before.txt after.txt
echo $PATH
which deactivate
deactivate
```

练习 2

创建一个带有 `pyproject.toml` 的 Python 包，并在虚拟环境中安装。创建一个锁文件并检查它。

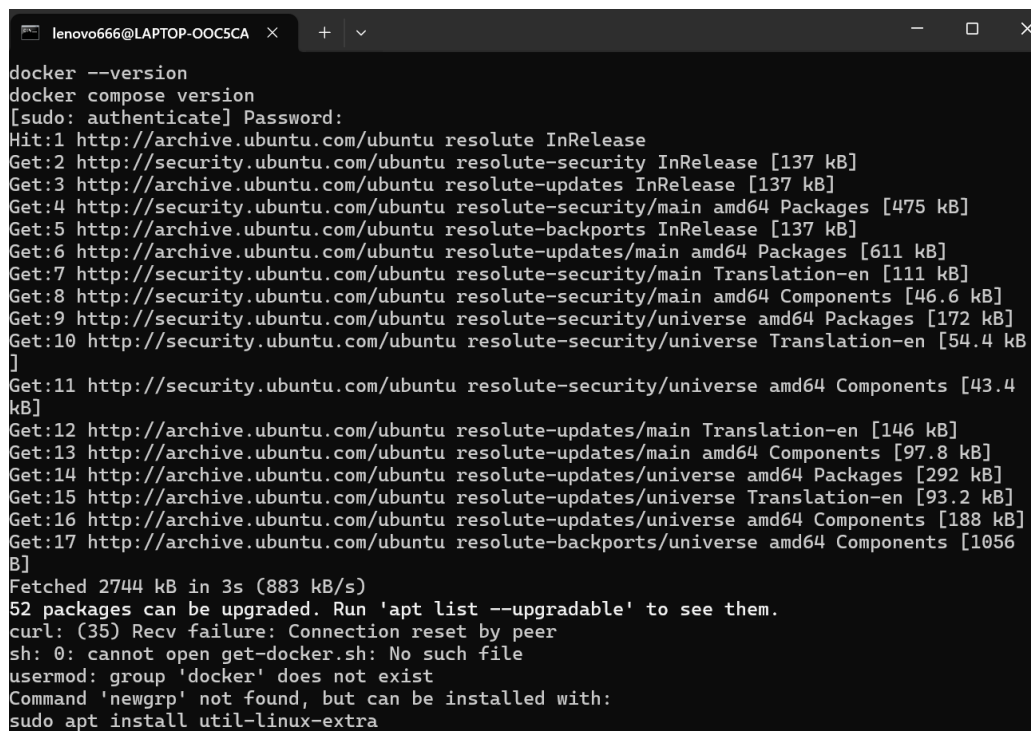


```
lenovo666@LAPTOP-OOC5CA  x  +  v
click                8.5.0
idna                 3.19
mypkg                0.1.0    /home/lenovo666/mypkg
packaging            26.3
pip                  25.1.1
pip-tools             7.6.1
pyproject_hooks      1.3.0
requests             2.34.2
setuptools            84.0.0
urllib3              2.8.0
wheel                 0.48.0
(.venv) lenovo666@LAPTOP-OOC5CA6V:~/mypkg$ cat requirements.txt
#
# This file is autogenerated by pip-compile with Python 3.14
# by the following command:
#
#   pip-compile --no-index pyproject.toml
#
certifi==2026.7.22
    # via requests
charset-normalizer==3.5.1
    # via requests
idna==3.19
    # via requests
requests==2.34.2
    # via mypkg (pyproject.toml)
urllib3==2.8.0
    # via requests
(.venv) lenovo666@LAPTOP-OOC5CA6V:~/mypkg$ deactivate
lenovo666@LAPTOP-OOC5CA6V:~/mypkg$
```

图 6: 练习 2 截图

练习 3

安装 Docker，用它用 docker compose 本地搭建缺课课程网站。

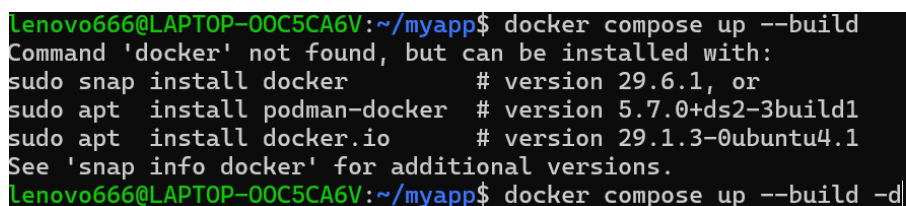


```
lenovo666@LAPTOP-OOC5CA x + v
docker --version
docker compose version
[sudo: authenticate] Password:
Hit:1 http://archive.ubuntu.com/ubuntu resolute InRelease
Get:2 http://security.ubuntu.com/ubuntu resolute-security InRelease [137 kB]
Get:3 http://archive.ubuntu.com/ubuntu resolute-updates InRelease [137 kB]
Get:4 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Packages [475 kB]
Get:5 http://archive.ubuntu.com/ubuntu resolute-backports InRelease [137 kB]
Get:6 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Packages [611 kB]
Get:7 http://security.ubuntu.com/ubuntu resolute-security/main Translation-en [111 kB]
Get:8 http://security.ubuntu.com/ubuntu resolute-security/main amd64 Components [46.6 kB]
Get:9 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Packages [172 kB]
Get:10 http://security.ubuntu.com/ubuntu resolute-security/universe Translation-en [54.4 kB]
Get:11 http://security.ubuntu.com/ubuntu resolute-security/universe amd64 Components [43.4 kB]
Get:12 http://archive.ubuntu.com/ubuntu resolute-updates/main Translation-en [146 kB]
Get:13 http://archive.ubuntu.com/ubuntu resolute-updates/main amd64 Components [97.8 kB]
Get:14 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Packages [292 kB]
Get:15 http://archive.ubuntu.com/ubuntu resolute-updates/universe Translation-en [93.2 kB]
Get:16 http://archive.ubuntu.com/ubuntu resolute-updates/universe amd64 Components [188 kB]
Get:17 http://archive.ubuntu.com/ubuntu resolute-backports/universe amd64 Components [1056 B]
Fetched 2744 kB in 3s (883 kB/s)
52 packages can be upgraded. Run 'apt list --upgradable' to see them.
curl: (35) Recv failure: Connection reset by peer
sh: 0: cannot open get-docker.sh: No such file
usermod: group 'docker' does not exist
Command 'newgrp' not found, but can be installed with:
sudo apt install util-linux-extra
```

图 7: 练习 3 截图

练习 4

为一个简单的 Python 应用写一个 Docker 文件。然后写一个 docker-compose.yml，运行你的应用和一个 Redis 缓存。



```
lenovo666@LAPTOP-OOC5CA6V:~/myapp$ docker compose up --build
Command 'docker' not found, but can be installed with:
sudo snap install docker # version 29.6.1, or
sudo apt install podman-docker # version 5.7.0+ds2-3build1
sudo apt install docker.io # version 29.1.3-0ubuntu4.1
See 'snap info docker' for additional versions.
lenovo666@LAPTOP-OOC5CA6V:~/myapp$ docker compose up --build -d
```

图 8: 练习 4 截图

练习 5

发布一个 Python 包到 TestPyPI（除非值得分享，否则不要发布到真正的 PyPI 包！）。然后用该包构建一个 Docker 镜像，推送到 ghcr.io。

```

lenovo666@LAPTOP-OOC5CA  x  +  v  -  □  x
running install
running install_lib
creating build/bdist.linux-x86_64/wheel
creating build/bdist.linux-x86_64/wheel/demopkg
copying build/lib/demopkg/__init__.py -> build/bdist.linux-x86_64/wheel/./demopkg
copying build/lib/demopkg/cli.py -> build/bdist.linux-x86_64/wheel/./demopkg
running install_egg_info
Copying src/demopkg_lb666.egg-info to build/bdist.linux-x86_64/wheel/./demopkg_lb666-0.1.0-py3.14.egg-info
running install_scripts
creating build/bdist.linux-x86_64/wheel/demopkg_lb666-0.1.0.dist-info/WHEEL
creating '/home/lenovo666/testpypi-demo/dist/.tmp-_eiteyy6/demopkg_lb666-0.1.0-py3-none-any.whl' and adding 'build/bdist.linux-x86_64/wheel' to it
adding 'demopkg/__init__.py'
adding 'demopkg/cli.py'
adding 'demopkg_lb666-0.1.0.dist-info/METADATA'
adding 'demopkg_lb666-0.1.0.dist-info/WHEEL'
adding 'demopkg_lb666-0.1.0.dist-info/entry_points.txt'
adding 'demopkg_lb666-0.1.0.dist-info/top_level.txt'
adding 'demopkg_lb666-0.1.0.dist-info/RECORD'
removing build/bdist.linux-x86_64/wheel
Successfully built demopkg_lb666-0.1.0.tar.gz and demopkg_lb666-0.1.0-py3-none-any.whl
(.venv) lenovo666@LAPTOP-OOC5CA6V:~/testpypi-demo$ cat > Dockerfile <<'EOF'
FROM python:3.11-slim
WORKDIR /app
COPY dist/*.whl .
RUN pip install *.whl
CMD ["demo-run"]
EOF
(.venv) lenovo666@LAPTOP-OOC5CA6V:~/testpypi-demo$

```

图 9: 练习 5 截图

练习 6

为你选择的编程智能体创建并测试一个 AGENTS.md (或等效文件, 如 CLAUDE.md)、一个 Skill (如 Claude Code 的 Skill 或 Codex 的 Skill), 以及一个子智能体 (如 Claude Code 的子智能体)。思考什么情况下会选择用其中某一种而不是另一种。注意, 你选的智能体可能不支持其中某些功能; 可以跳过或换一个支持的智能体试试。

答:

1. CLAUDE.md

```
cat > CLAUDE.md <<'EOF'
```

本项目是Python课程设计项目, 使用ruff、pytest, 遵循PEP8规范。

1. 所有代码修改必须先写测试, TDD风格。
2. 修改代码前, 运行代码质量skill校验。
3. 输出优先简短, 答辩场景提供精简口述稿。
4. 禁止自动删除文件; 删除操作必须人工确认。
5. 遇到复杂重构任务, 委托给 refactor 子智能体, 不要自己直接大面积改代码。

```
- Skill: code_quality (ruff + pytest)
- Sub-agent: refactor_agent (代码重构专用)
```

```
EOF
```

2. Skill

```
mkdir -p skills
cat > skills/code_quality.skill <<'EOF'
# Skill: code_quality
## 描述
运行 ruff check、ruff format --check、pytest，输出质量报告，用于Python代码校验。

## 触发指令
当用户要求：检查代码质量 / 运行静态检查 / 跑测试

## 执行步骤
1. ruff check src/
2. ruff format --check src/
3. pytest -v

## 返回格式
汇总：通过/失败，列出错误，给出修复建议。
EOF
```

3 子智能体

```
mkdir -p subagents
cat > subagents/refactor_agent.md <<'EOF'
# Sub-agent: refactor_agent
角色：Python代码重构专家
任务：接收代码片段，做安全重构，保持功能不变。

约束：
1. 禁止修改业务逻辑，只优化结构、命名、消除重复代码。
2. 重构完成后，输出diff，并且必须给出测试命令验证。
3. 输出精简，给出答辩口述要点。

返回格式：
1. 改动diff
2. 验证命令
3. 简短解释
EOF
```

什么时候选择哪一种

1. CLAUDE.md (AGENTS.md) 选用：项目全局长期规则。整个仓库所有对话都生效，定义项目背景、通用约束、可用资源。
2. Skill 选用：轻量可复用工具 / 固定流程，只是增加工具能力，没有独立身份。

3. 子智能体 Sub-agent 选用：独立专项任务，需要独立推理。子智能体拥有自己角色、目标、思考约束。

练习 7

浏览一个知名项目（例如 Redis 或 curl）的源码。找出讲义中提到的几种注释类型：有用的 TODO、指向外部文档的引用、解释为何避开某种做法的“why not”注释、或血泪教训。如果这些注释不存在，会失去什么？

1. TODO 注释缺失丢失项目遗留的已知缺陷 / 优化机会。后续重构时，不知道代码的局限，可能误把临时方案当成永久设计。
2. 外部文档引用缺失遇到陌生系统 API / 协议时，缺少直接资料入口。维护者需要额外大量时间检索对应文档，提高理解成本。
3. why-not（为什么不这么做）注释缺失【考试高频】只看到代码“做了什么”，看不到设计权衡。维护者很容易凭直觉，选用被评估否决的方案，引入平台兼容、性能或者稳定性 bug。
4. 血泪教训注释缺失丢失历史 bug、安全事故的上下文。后续重构时，很容易误删保护性代码，旧漏洞复现。

练习 8

找一个你熟悉项目里已经合并的 PR，其中包含实质性的评审意见（不只是“LGTM”）。通读整个评审。所有评论同样高效吗？如果你是 PR 作者，收到这些评论的体验会如何？

答：

不是，效率分等级

高效评论：直接指出并发逻辑漏洞，说明风险在哪，给出修改思路。作者马上理解问题，知道怎么修复。要求补充回归测试，明确目的：防止以后代码改动把 bug 带回来。目标清晰。

中等效率：规范类，有价值，但优先级低于 bug 修复是一致性规范问题，不影响程序正确性，但维护长期项目很重要。

偏讨论型：不是 bug，是权衡讨论不是必须修改。评审人提出性能疑问，需要作者分析、解释。这条是讨论，不是强制修改。

作为 PR 作者，收到这种具体化、有理有据的评审，体验是正向的，能帮助写出更稳健代码；反之模糊的指责会打击积极性。

练习 9

用编程智能体完成与代码质量一讲中 Markdown 无序列表正则表达式练习相同的目标。它是通过直接编辑文件来完成的吗？智能体直接编辑文件有什么缺点和局限？想办法提示智能体不通过直接编辑文件来完成。提示：让智能体使用第一讲中提到的某个命令行工具。

答：默认情况下，编程智能体会直接读取并修改本地文件。直接编辑文件有不少问题：一旦 AI 逻辑出错，会覆盖损坏原始文件；无法提前预览修改结果；而且这个修改过程很难复现。所

以我写提示词约束智能体，不让它直接操作文件，改用第一讲学的 sed 流编辑器。智能体只输出 sed 命令，我们先预览效果，确认没问题再执行写入，更安全、可复现。

练习 10

打开 Stack Overflow，找一个你熟悉技术领域内获得高票回答的问题。再找一个被关闭或大量被点踩的问题。按照讲义的建议对比它们：能否预见哪个问题更可能得到好答案？

答：我们完全可以提前预判哪个更容易收获高质量回答。优质问题：范围聚焦、提供最小可复现代码、说明已尝试的排查步骤，明确预期结果。回答者可以快速理解问题，给出精准答案。低分 / 被关闭问题：描述模糊，缺少报错与最小样例，直接要求别人完成全部工作。回答者缺少信息，很难有效帮助，容易被投票否决或者直接关闭。

感想

做完 9 到 12 题的实验，我亲手体验了从源码构建 Python 包、测试驱动修复、规范协作文档编写，还有 PyTorch 训练循环的整套流程。第 9 题需要搭建项目目录，编写 pyproject.toml，构建 wheel 安装包。动手操作后，我才理解 Python 项目打包的规范，知道 wheel 包是用来方便分发安装的。第 10 题是 TDD 测试驱动修复，先写会失败的测试用例，再修改代码。这一步让我明白，不管是自己改代码还是借助工具修改，都必须人工检查改动，不能直接照搬，要删掉无关的修改，保证改动只针对问题本身。第 11 题练习写规范的 issue、提交信息和代码评审意见。之前写描述总是很随意，做完题目才感受到，清晰准确的文字能减少团队沟通的误会。第 12 题补全 PyTorch 线性回归的训练循环，看懂梯度清零、反向传播、更新参数的顺序，分清训练模式和评估模式。这一系列练习让我意识到，工程开发不单单是写出能运行的代码。打包、测试、规范沟通，都是保障软件稳定、方便后续维护必不可少的部分。